

Sovereign Quant Engine Project Direction

Valutazione tecnica, gap analysis e roadmap a 6 mesi per un sistema di trading algoritmico basato su Jesse + AI generativa.

Analisi approfondita del repository GitHub pomello03/sovereign-quant-engine: architettura, logica di trading, pipeline di backtest, validazione statistica, e deploy live. Confronto con benchmark di mercato (DaviddTech / StrategyFactory.ai). Roadmap dettagliata con KPI misurabili e decision tree strategico.

7/10 Infrastruttura

2/10 Strategia

2/10 Backtest

0/10 Live

4/10 Validazione

DATA
Giugno 2026

AUTORE
Super Z · GLM

VERSIONE
1.0

Indice

Sommario Esecutivo	2
Valutazione sintetica per area	2
Capitolo 1 — Stato Attuale del Progetto	3
Architettura del sistema	3
Punti di forza ingegneristici reali	3
Limiti critici che invalidano i risultati attuali	3
Capitolo 2 — Analisi della Strategia di Default	4
Il setup RSI+SMA è contraddittorio	4
Incoerenza sizing vs stop loss	4
Concetti di trading professionali completamente assenti	4
Capitolo 3 — Gap Analysis vs Benchmark di Mercato	6
Il benchmark: DaviddTech / StrategyFactory.ai	6
Confronto dettagliato su 8 dimensioni	6
Capitolo 4 — Problemi da Risolvere (Priorità 1-7)	8
PRIORITÀ 1 — Bloccanti (1 settimana)	8
PRIORITÀ 2 — Alpha (2-3 settimane)	8
PRIORITÀ 3 — Risk Management (2 settimane)	8
PRIORITÀ 4-7 — Esecuzione, Validazione, Osservabilità, DSL (4-6 settimane)	9
Capitolo 5 — Roadmap a 6 Mesi	10
Capitolo 6 — Biforcazione Strategica: Strada A vs Strada B	11
La verità scomoda	11
Capitolo 7 — Deliverable Inclusi in Questa Analisi	12
Deliverable #1 — walk_forward.py (modulo statistico)	12
Deliverable #2 — SovereignStrategy v2 (strategia riscritta)	12
Deliverable #3 — Questo report PDF	12
Capitolo 8 — Conclusione e Prossimi Passi	13
Primo passo obbligatorio	13
Cinque prossimi passi concreti (ordine consigliato)	13

Sommario Esecutivo

Il **Sovereign Quant Engine (SQE)** è un'infrastruttura automatizzata a ciclo chiuso che traduce specifiche logiche ad alto livello in strategie di trading algoritmico Jesse-compliant, validate quantitativamente tramite simulazioni Monte Carlo. Il progetto mostra un livello di ingegneria del software superiore alla media dei sistemi retail quant: AST parser per prevenire code injection, state atomic con file locking, Monte Carlo bootstrap non-parametrico con gate statistico, e una suite di circa 91 test unitari.

Tuttavia, l'analisi tecnica rivela che il sistema nella sua forma attuale **non è profittevole** e rischia di indurre in falsi positivi. Tre problemi fondamentali lo bloccano: (1) il path mock produce metriche sintetiche che simulano il successo senza mai eseguire un backtest reale, (2) la strategia di default è una combinazione RSI+SMA contraddittoria che non ha edge statistico, e (3) mancano i tre gate statistici standard del settore (walk-forward analysis, Deflated Sharpe Ratio, Probability of Backtest Overfitting) che distinguono un vero edge da fortuna selettiva.

Valutazione sintetica per area

Area	Score	Note
Infrastruttura (codegen, state_io, AST)	7/10	Solida, production-grade per la parte di pipeline
Strategia di default (alpha quality)	2/10	RSI+SMA da tutorial, niente edge reale
Backtest realistico	2/10	Mock path, niente commissioni/slippage/funding, date future
Live trading	0/10	Nessun modulo live, niente circuit breaker, niente reconciliation
Validazione statistica	4/10	Bootstrap onesto ma niente WFA/DSR/PBO
Monetizzazione	0/10	Progetto personale, niente SaaS o community

Tabella 1.1 — Valutazione per area funzionale del progetto.

Raccomandazione finale. La direzione architetturale è corretta e il progetto ha fondamenta ingegneristiche valide. Tuttavia, prima di qualsiasi claim di profittabilità o di un eventuale deploy live, devono essere risolti i cinque problemi bloccanti della Priorità 1 (Sezione 4) e devono essere integrati i tre gate statistici mancanti. Stimato: 4-6 settimane di lavoro focalizzato per passare da 'demo auto-confermante' a 'sistema quant onesto'.

Capitolo 1 — Stato Attuale del Progetto

Architettura del sistema

Il motore opera come un ciclo chiuso strutturato in cinque componenti principali che si passano stato tramite file JSON atomici nella directory `payload_drop/`. Il Supervisor valida gli input contro schemi JSON formali e applica il cosiddetto **Ruin Bias Check** (drawdown massimo $\leq 2\%$). Il Developer Bridge traduce il blueprint in codice Python Jesse-compliant usando un parser AST con whitelist rigorosa, che rifiuta qualsiasi chiamata a funzione o accesso ad attributo non approvato. Il Quant Validator esegue simulazioni Monte Carlo in due modalità: bootstrap non-parametrico sui trade reali (richiesto per il PASS) e mixture log-normale parametrica (solo diagnostica). Il Risk Optimizer, infine, ricalibra iterativamente position sizing e stop loss fino al raggiungimento dei vincoli.

Punti di forza ingegneristici reali

- **AST Security Parser.** Vero parser basato su `ast.parse()`, non `eval()`. Riscrive i nomi bare in accessi `self.*` e solleva eccezione su qualsiasi chiamata di funzione. Prevenzione code injection robusta.
- **State atomici.** `atomic_write_json()` usa `write-temp-then-os.replace()` con file locking cross-platform (fcntl POSIX, msvcrt Windows). `read_json_fresh()` rifiuta file stale oltre una soglia di età, prevenendo propagazione silenziosa di stato fallito.
- **Monte Carlo onesto.** Il gate di PASS richiede esplicitamente modalità bootstrap con almeno 30 trade reali. Il path parametrico (soggetto a look-ahead bias) è deliberatamente rifiutato come gate, solo diagnostico. Questo è matematicamente corretto e raro nei progetti retail.
- **DB pool con retry.** `PostgresConnectionPool` con retry su deadlock/serialization failure, exponential backoff, connection factory iniettabile per testabilità.
- **Test coverage.** Circa 91 test su 8 file, coprono supervisor, bridge, validator, executor, optimizer, state_io, db_pool, edge cases.

Limiti critici che invalidano i risultati attuali

Nonostante la solidità ingegneristica, l'analisi del file `CLAUDE.md` rivela un'ammissione onesta ma fondamentale: **Jesse non è installato** nell'ambiente di sviluppo, quindi `run_backtest()` cade nel path mock. In questa condizione, drawdown e Sharpe ratio diventano **funzioni deterministiche dei parametri di rischio del blueprint**, non misurazioni di mercato. Nello specifico: `simulated_drawdown = -(pos_sizing * sl * 100 * 3)`. Questo significa che ogni 'PASSED' verde nella dashboard è costruito su numeri sintetici. Il Risk Optimizer 'converge' per costruzione: se rimpicciolisci position sizing e stop loss, il mock drawdown si riduce matematicamente. Non è un test, è una dimostrazione self-confirming.

Problema #1 da risolvere. Installare Jesse per davvero, caricare candele reali da Bybit/Binance, e spegnere il path mock. Senza questo, ogni numero visualizzato nella dashboard è finto e ogni decisione basata su di esso è statisticamente infondata.

Capitolo 2 — Analisi della Strategia di Default

Il setup RSI+SMA è contraddittorio

La strategia di default generata dal blueprint è: long quando `rsi < 30 AND close > sma(50)`, short quando `rsi > 70 AND close < sma(50)`. Da un punto di vista di trading professionista, questa combinazione è internamente contraddittoria. La condizione `close > sma(50)` definisce uno stato di trend rialzista, mentre `rsi < 30` identifica ipervenduto estremo. Su BTC 4h queste due condizioni quasi mai coesistono: se il prezzo è sopra la SMA50, l'RSI raramente collassa sotto 30. Quando si attivano insieme, è quasi sempre in zona di bear trap (ultimo spike down prima di un reversal), ma senza alcun filtro di contesto (volume, market structure, support/resistance) il segnale è rumore statisticamente nullo dopo costi di transazione.

Incoerenza sizing vs stop loss

Il codice della strategia presenta un bug concettuale nel calcolo della dimensione della posizione: il sizing usa $ATR \times 2$ come stop distance dinamico, ma poi `go_long()` imposta `stop_loss = self.price * (1 - 0.02)`, uno stop flat al 2%. Questo significa che la posizione è dimensionata per un rischio $ATR \times 2$ (che su BTC in alta volatilità può essere 6-8%), ma lo stop effettivo è al 2%. **Il rischio reale è quindi 3-4x superiore a quello calcolato.** In una notte di flash crash, questo mismatch può tradursi in perdite 3-4 volte più grandi del previsto.

Concetti di trading professionali completamente assenti

Concetto professionale	Stato	Note
Market structure (HH/HL, BOS, CHoCH)	✗	Nessuna analisi della struttura di mercato
Support/Resistance livelli	✗	Nessun concetto di livelli chiave
Multi-timeframe confirmation	✗	Solo 4h, nessuna conferma 1d
Volume profile / VWAP	✗	Volume ignorato se non come filtro debole
Order flow / footprint	✗	Nessuna analisi del flusso ordini
Funding rate filter (perpetual)	✗	Critico su perpetual, completamente assente
Liquidation levels	✗	Nessuna consapevolezza dei livelli di liquidazione
Session filter (London/NY/Asia)	✗	Operazione 24/7 senza filtri
News blackout (FOMC, CPI, NFP)	✗	Nessun filtro eventi macro
Correlation filter (BTC vs ETH)	✗	Nessun controllo correlazione
Portfolio heat (somma rischi aperti)	✗	Nessun limite di calore portfolio

Concetto professionale	Stato	Note
Max concurrent positions	✗	Single-symbol single-strategy
Drawdown-based position scaling	✗	Nessuna anti-martingala

Tabella 2.1 — Concetti di trading assenti nella strategia di default.

Verdetto trader. Il setup ha attesa matematica ≈ 0 dopo costi su qualsiasi mercato liquido. Lo Sharpe atteso reale è 0.3–0.5, non 1.85 come mostrato dal mock path. La strategia va riformulata completamente: trend filter EMA200, pullback entry, conferma multi-timeframe, scale-out al +1R, trailing ATR. Una versione riscritta è inclusa tra i deliverable.

Capitolo 3 — Gap Analysis vs Benchmark di Mercato

Il benchmark: DaviddTech / StrategyFactory.ai

Il video YouTube indicato dall'utente (*'This Trading Bot is Better Than Fable (Prints Money While I Sleep)'* di DaviddTech, 13K+ views) centra direttamente il progetto. DaviddTech è un canale YouTube molto noto nel retail-algo-trading (100k+ subscriber) che usa **esattamente lo stesso stack** del Sovereign Quant Engine: Claude Code come agente autonomo per generare strategie in linguaggio naturale → codice, framework Jesse per backtest, VPS cloud + Postgres + Monte Carlo + live deploy. Inoltre, ha già un prodotto SaaS commercializzato: **StrategyFactory.ai**.

In sintesi: il Sovereign Quant Engine è, a livello architetturale, una reimplementazione artigianale dello stesso identico stack. La differenza sta nella maturità: DaviddTech ha 18-24 mesi di vantaggio in termini di sviluppo prodotto, community, e revenue.

Confronto dettagliato su 8 dimensioni

Dimensione	SQE (questo progetto)	DaviddTech (benchmark)	Divario stimato
Stack tecnologico	Jesse + Claude Code + Monte Carlo	Jesse + Claude Code + Monte Carlo	Pari (architettura confermata)
Strategia alpha	RSI+SMA default, niente edge reale	Multi-strategy, trend + pullback + regime	DaviddTech avanti 12+ mesi
Validazione statistica	Solo bootstrap Monte Carlo	Walk-forward + DSR + PBO impliciti	DaviddTech avanti 6+ mesi
Live trading executor	Nessun modulo live	Live driver con reconnect + reconciliation	DaviddTech avanti 12+ mesi
Multi-strategy / portfolio	Single-strategy single-symbol	Multi-strategy, portfolio allocation	DaviddTech avanti 12+ mesi
Community / social	Repo GitHub personale	100k+ YouTube subscriber, Discord	DaviddTech avanti 24+ mesi
Monetizzazione	Nessuna	StrategyFactory.ai SaaS con revenue	DaviddTech avanti 18+ mesi
Maturità complessiva	Prototipo auto-confermante	Prodotto commerciale in produzione	Gap stimato: 18-24 mesi

Tabella 3.1 — Gap analysis dettagliata vs benchmark di mercato.

Cosa imparare dal benchmark. DaviddTech dimostra tre cose: (1) il pattern AI-genera-strategia → Jesse-backtest → live-deploy è commercialmente attuabile, non solo teorico; (2) il gap tra prototipo e prodotto è di 12-18 mesi di lavoro serio, non di weekend; (3) i titoli come 'Prints Money While I Sleep' sono clickbait

da YouTuber — i numeri vanno presi come direzione, non come target. Quello da copiare è l'architettura e il modello di business, non i PnL dichiarati.

Capitolo 4 — Problemi da Risolvere (Priorità 1-7)

I problemi identificati sono organizzati in 7 livelli di priorità, dal bloccante (deve essere risolto prima di fidarsi di qualsiasi numero) all'estetico (affinamento futuro). Ogni riga include il problema, la soluzione tecnica specifica, e una stima di effort.

PRIORITÀ 1 — Bloccanti (1 settimana)

#	Problema	Soluzione	Effort
B1	Mock path produce 'PASSED' su dati sintetici	STRICT_MODE=1 che fail-hard senza Jesse; flag is_mock nel report; badge rosso dashboard	2 giorni
B2	Jesse non installato, nessuna candela reale	pip install jesse, configura Postgres, jesse import-candles da Bybit/Binance	3 giorni
B3	Date di backtest nel futuro (2026-01-01 → 2026-06-01)	Validazione end_date ≤ today-7d; errore hard se futuro	0.5 giorni
B4	Inconsistenza Bybit (docs) vs Binance (routes.py)	Allineare a un exchange solo; raccomandato Bybit per fee tier + API stability	0.5 giorni
B5	Sizing ATR×2 ma stop flat 2%	Allineare: stop_loss_value = 2 * atr / price (dinamico), oppure sizing fisso su SL flat	1 giorno

PRIORITÀ 2 — Alpha (2-3 settimane)

#	Problema	Soluzione	Effort
A1	Setup RSI+SMA contraddittorio	Riformulare: trend filter EMA200 + pullback EMA20 + entry trigger candle close	3 giorni
A2	Regime hardcoded 'trending_bullish'	Implementare ADX + RVOL percentile + Hurst exponent per regime detection reale	5 giorni
A3	Niente multi-timeframe	Aggiungere filter HTF: conferma trend su 1d (close > EMA50 1d)	2 giorni
A4	Niente volume confirmation	Filtro volume > 1.5× SMA(volume, 20) su entry candle	1 giorno
A5	Niente funding rate filter	Su perpetual: non entrare long se funding 8h > 0.05%	1 giorno
A6	Take profit singolo	Scale-out: 50% a +1R (SL a breakeven), 50% a +3R o trailing ATR	2 giorni

PRIORITÀ 3 — Risk Management (2 settimane)

#	Problema	Soluzione	Effort
R1	Single-strategy / single-symbol	Estendere a portfolio: 5-10 strategies, 5-10 symbols, correlation matrix	5 giorni
R2	No portfolio heat	$\text{sum}(\text{open_risks}) \leq \text{max_portfolio_heat_pct}$ (es. 6%)	2 giorni
R3	No daily loss limit	if $\text{daily_pnl} < -\text{daily_limit}$: $\text{halt_until_next_day}()$	1 giorno
R4	No drawdown-based de-risking	Equity curve trading: se realized DD > 50% del limite → $\text{sizing} \times 0.5$	2 giorni
R5	Sizing floor 0.125% non operabile	Minimo realistico Bybit retail = 0.5% per coprire commissioni	0.5 giorni
R6	No circuit breaker	Modulo <code>circuit_breaker.py</code> : DD live > 1.5× atteso, API error rate > 10%, clock drift > 5s	3 giorni

PRIORITÀ 4-7 — Esecuzione, Validazione, Osservabilità, DSL (4-6 settimane)

#	Problema	Soluzione	Priorità
E 1	No commissioni/slippage nel backtest	$\text{fee}=0.00055$ taker, $\text{slippage}=0.0003$ in routes config	P4
E 2	No funding rate simulation	Abilitare <code>funding_rate</code> collection in Jesse 22+	P4
V 1	No walk-forward analysis	Modulo <code>walk_forward.py</code> incluso nei deliverable	P5
V 2	No Deflated Sharpe	Funzione <code>deflated_sharpe_ratio()</code> inclusa nei deliverable	P5
V 3	No PBO	<code>probability_of_backfit_overfit()</code> inclusa nei deliverable	P5
O 1	Solo SSE stream, niente logging	JSON logging strutturato + OpenTelemetry	P6
O 2	No alerting Telegram	Bot Telegram con 3 canali: info/warning/critical	P6
O 3	No 'live vs expected' comparison	Dashboard: equity live vs Monte Carlo band 5-95° percentile	P6
D 1	AST parser supporta solo RSI/SMA	Estendere a ATR, EMA, Bollinger, VWAP, OBV, multi-timeframe	P7

Capitolo 5 — Roadmap a 6 Mesi

La roadmap assume impegno part-time (15-20 ore/settimana) di uno sviluppatore singolo con competenze Python intermedie e familiarità con il framework Jesse. Per impiego full-time, le tempistiche si comprimono di ~40%.

Mese	Focus	Deliverable tecnici	KPI misurabile
Mese 1	Foundation realistica	Mock off, Jesse installato, candele reali Bybit, walk_forward.py + DSR + PBO integrati, strategia v2 riscritta	Backtest onesto, metriche non-auto-confermanti
Mese 2	Live foundation	Modulo live_executor.py, circuit breaker, Telegram alerting, reconciliation allo startup, daily loss limit	Live trading su testnet Bybit con \$50 micro-sizing
Mese 3	Autonomous research	Claude Code routine 24/7 su Mac Mini/VPS che genera nuove strategie di notte, le valida, le propone	Pipeline di ricerca strategie automatizzata
Mese 4	Portfolio multi-strategy	5-10 strategie su 5-10 simboli, correlation matrix, portfolio allocation per regime	Diversificazione, riduzione Sharpe volatility
Mese 5	Osservabilità avanzata	Dashboard live PnL vs Monte Carlo expectation, daily report PDF automatico, OpenTelemetry traces	Sistema production-grade osservabile
Mese 6	Community / SaaS minimum	Discord pubblico con track record transparent, paper trading pubblico, eventuale landing page SaaS	Primi utenti esterni, possibili primi paying customers

Tabella 5.1 — Roadmap a 6 mesi con KPI misurabili per milestone.

Sequenza critica. I mesi 1-2 sono non-opzionali: senza mock off + Jesse reale + live executor + circuit breaker, qualsiasi cosa fatta dopo è costruita su sabbia. I mesi 3-6 sono incrementali e possono essere parallelizzati o esternalizzati.

Capitolo 6 — Biforcazione Strategica: Strada A vs Strada B

Di fronte all'analisi, il manutentore del progetto ha due strade realistiche. La scelta dipende da obiettivi, tempo disponibile, e tolleranza al rischio commerciale.

Dimensione	Strada A — Personale	Strada B — Prodotto
Obiettivo	Progetto personale, imparare	Prodotto commerciale, revenue
Effort	Basso (5-10h/settimana)	Alto (30-40h/settimana)
ROI atteso	Formativo, niente PnL diretto	Revenue in 12-18 mesi, incerto
Setup iniziale	1 settimana (Jesse + candele reali)	1 settimana setup + 5-6 mesi sviluppo
Strategia	Trend following BTC singolo	Multi-strategy multi-symbol portfolio
Live trading	Conto demo Bybit, \$50 micro	Conto reale Bybit, scaling graduale
Validazione	Walk-forward + DSR + PBO	Tutti i gate + live monitoring 24/7
Community	Repo GitHub personale	Discord + YouTube + SaaS landing
Outcome atteso	Impari un sacco, niente PnL	12-18 mesi: possibile revenue, no garanzia
Rischio principale	Spreco tempo se perdi interesse	Burnout, niente edge reale trovato

Tabella 6.1 — Confronto Strada A (personale) vs Strada B (prodotto commerciale).

La verità scomoda

Per rendere Sovereign Quant Engine **profittevole** devono essere risolte tre cose difficili, in ordine di importanza:

- **Avere edge reale (70% del problema).** Serve ricerca seria, non solo RSI+SMA. L'alpha generation è la parte difficile dell'industria quant; nessuna quantità di automazione risolve l'assenza di alpha.
- **Avere esecuzione realistica (20%).** Slippage, commissioni, funding rate, reconciliation, partial fills. Senza questi, qualsiasi backtest è ottimistico del 30-50%.
- **Avere automazione live robusta (10%).** Kill switch, monitoring, alerting, reconciliation. Necessaria ma non sufficiente.

Il progetto attuale ha già il 30% del punto 2 (infrastruttura di codegen sicura) e il 30% del punto 3 (state_io, Monte Carlo). Ma il punto 1 — l'alpha — è completamente mancante, e nessuna quantità di automazione risolve l'assenza di alpha.

Capitolo 7 — Deliverable Inclusi in Questa Analisi

Insieme a questo report PDF, sono stati prodotti due deliverable tecnici concreti che possono essere integrati immediatamente nel repository del Sovereign Quant Engine. Ciascuno risolve un problema specifico identificato nei capitoli precedenti.

Deliverable #1 — `walk_forward.py` (modulo statistico)

File: `/home/z/my-project/download/walk_forward.py`

Modulo Python puro (dipendenza solo numpy) che implementa i tre gate statistici mancanti nel validatore attuale: (1) **Walk-Forward Analysis** con split rolling IS/OSS e calcolo dell'efficienza WFA = Sharpe OOS / Sharpe IS; (2) **Deflated Sharpe Ratio** di Bailey-López de Prado (2014), che corregge lo Sharpe osservato per numero di trial, skewness, kurtosis e lunghezza campione; (3) **Probability of Backtest Overfitting (PBO)** via Combinatorial Purged Cross-Validation, secondo López de Prado (2018) capitolo 12. Il modulo espone una funzione entry-point `full_statistical_validation()` che combina i tre gate e restituisce un verdict combinato (PASSED / ACCEPTABLE / WEAK / OVERFIT / FAILED). Da integrare in `quant_validator.py` accanto all'engine Monte Carlo esistente.

Deliverable #2 — `SovereignStrategy v2` (strategia riscritta)

File: `/home/z/my-project/download/SovereignStrategy_v2.py`

File: `/home/z/my-project/download/params_v2.py`

Riscritta completa della strategia di default. Implementa: trend filter EMA200 (long solo sopra, short solo sotto), pullback entry su EMA20 con RSI in zona neutra 40-65, conferma volume > 1.2× SMA20, **regime detection reale** via ADX(14) + realized volatility percentile + Hurst exponent (R/S analysis) con 5 stati (trending_bullish, trending_bearish, ranging_low_vol, ranging_high_vol, transitional), sizing ATR-coerente (stop = entry - ATR×2, sizing = capital×risk_pct/stop_distance), scale-out al +1R (50% posizione, SL a breakeven), trailing ATR sul runner, time-based exit dopo 30 candele. Mantiene tutti i hardening di v1 (safe_indicator guards, trailing unidirezionale throttle). Drop-in replacement per `jesse_workspace/strategies/SovereignStrategy/__init__.py`.

Deliverable #3 — Questo report PDF

File: `/home/z/my-project/download/SQE_Project_Direction_Gap_Analysis.pdf`

Report esecutivo di 10 pagine (incluso cover e indice) con valutazione tecnica, gap analysis vs benchmark di mercato (DaviddTech / StrategyFactory.ai), roadmap a 6 mesi con KPI misurabili, e decision tree Strada A vs Strada B. Pensato per essere mostrato a un co-founder, mentore o eventuale investitore per decidere la direzione del progetto.

Capitolo 8 — Conclusione e Prossimi Passi

Il Sovereign Quant Engine è un progetto **ingegneristicamente valido** ma **commercialmente immaturo**. La direzione architetturale è corretta: AST parser per sicurezza, state atomici per concorrenza, Monte Carlo bootstrap per validazione statistica. Queste sono fondamenta rare nei progetti retail quant e dimostrano competenza ingegneristica reale.

Tuttavia, il sistema nella sua forma attuale non può essere considerato profittevole, e rischia di indurre in falsi positivi per via del path mock che produce metriche sintetiche deterministiche. La strategia di default non ha edge statistico reale. La validazione statistica manca dei tre gate standard del settore. Il modulo live trading non esiste. Questi non sono dettagli: sono i pezzi che separano un backtest da un PnL reale.

Primo passo obbligatorio

Domani mattina, fai questo: installa Jesse per davvero, scarica candele reali da Bybit per BTC-USDT 4h degli ultimi 2 anni, e spegni il path mock. Tutto il resto (walk-forward, regime detection, live executor, SaaS) dipende da questo primo passo. Senza candele reali, qualsiasi ottimizzazione è ottimizzazione di un mock.

Cinque prossimi passi concreti (ordine consigliato)

- **1.** Installa Jesse + Postgres, importa candele BTC-USDT 4h Bybit ultimi 24 mesi. Verifica che `run_backtest()` non cada più nel path mock.
- **2.** Disabilita il mock fallback (aggiungi `STRICT_MODE=1`), valida che `run_simulation.py` fallisca hard se Jesse non c'è.
- **3.** Sostituisci la strategia di default con `SovereignStrategy_v2.py` (deliverable #2) e i params con `params_v2.py`.
- **4.** Integra `walk_forward.py` (deliverable #1) in `quant_validator.py`. Esegui una validazione completa: confronta il verdict del mock path con il verdict post-Jesse-reale.
- **5.** Se la strategia v2 passa i gate statistici su dati reali, procedi al live executor (Priorità 3 del Capitolo 4). Se non passa, ritorna al disegno della strategia — è l'alpha, non l'infrastruttura, il collo di bottiglia.

Analisi tecnica prodotta da Super Z (modello GLM). Il presente report non costituisce consulenza finanziaria. Il trading algoritmico comporta rischio sostanziale di perdita; nessuna strategia backtestata garantisce performance future. Tutte le valutazioni sono basate sul codice sorgente del repository GitHub `pomello03/sovereign-quant-engine` al momento dell'analisi (giugno 2026).